Методические материалы по лабораторным работам курса “Fullstack”

Общие требования. 2

Лабораторная работа №1 - Проектирование и реализация модели ролей и прав доступа в веб-приложении. 3

Лабораторная работа №2 - Аутентификация и авторизация пользователей в веб-приложении на основе access и refresh токенов с применением архитектурных и проектных паттернов. 5

Лабораторная работа №3 - Разработка пользовательских интерфейсов с фильтрацией и управление пользовательскими данными с интеграцией объектного хранилища. 7

Лабораторная работа №4 - SEO-оптимизация веб-приложения и интеграция сторонних API. 9

Лабораторная работа №5 - Комплексное тестирование клиентской и серверной частей веб-приложения. 11

Лабораторная работа №6 - Контейнеризация и автоматизация развертывания веб-приложения. 13

Общие требования.

Используемые технологии(использование других технологий только по согласованию с преподавателем):

1. Backend: Python + FastAPI
2. Frontend: TypeScript + React
3. Введение Git.

Лабораторная работа №1 - Проектирование и реализация модели ролей и прав доступа в веб-приложении.

Цели лабораторной работы

1. Ознакомиться с принципами управления доступом: RBAC, принцип минимальных привилегий, запрет по умолчанию.
2. Научиться проектировать роли и разрешения для уже существующего MVP.
3. Реализовать проверку прав на backend (FastAPI) и ролевое поведение интерфейса на frontend (React + TypeScript).
4. Проверить корректность и устойчивость работы приложения после доработки функционала доступа.

Задание

1. Возьмите **существующий MVP** (разработанный в прошлом семестре) и проанализируйте текущую структуру сущностей и пользовательских сценариев.
   Определите, какие сущности и действия в вашем проекте должны быть ограничены правами доступа (например: просмотр, создание, редактирование, удаление, публикация, модерация и т.д.).

2. Спроектируйте модель ролей и прав доступа для вашего текущего проекта. Для этого:

   2.1. выделите минимум **2-3 роли** (например: guest, user, manager, admin);
   2.2. определите список разрешений (permissions) для каждой роли;
   2.3. задайте дополнительные ограничения (например, доступ только к собственным данным, запрет удаления для отдельных ролей);
   2.4. подготовьте матрицу ролей и прав (role → allowed actions).

3. Доработайте backend в **FastAPI** для поддержки RBAC:

   3.1. добавьте/расширьте модели данных для ролей и прав (или эквивалентные структуры);
   3.2. реализуйте проверку прав на уровне endpoint'ов (dependencies/guards/decorators);
   3.3. обеспечьте корректный отказ в доступе при недостатке прав (403 Forbidden);

3.4. реализуйте минимум один endpoint для управления ролями пользователей (доступен только администратору).

4. Доработайте frontend в **React + TypeScript** с учетом ролей:

4.1. адаптируйте интерфейс под права текущего пользователя (скрытие/блокировка недоступных действий);
4.2. внедрите защиту маршрутов (приватные и ролевые роуты);

5. Убедитесь, что после внедрения ролевой модели:

5.1. существующая функциональность MVP не нарушена;
5.2. права доступа применяются стабильно для всех ролей;
5.3. неавторизованные и недостаточно привилегированные действия блокируются корректно.

Лабораторная работа №2 - Аутентификация и авторизация пользователей в веб-приложении на основе access и refresh токенов с применением архитектурных и проектных паттернов.

Цели лабораторной работы

1. Освоить практику построения безопасной схемы аутентификации на основе access и refresh токенов.
2. Научиться отделять ответственность между слоями приложения с применением архитектурных и проектных паттернов.
3. Реализовать согласованную авторизацию на сервере и клиенте с учетом ролей и прав доступа.
4. Обеспечить устойчивую работу механизма сессий в существующем MVP без нарушения текущей функциональности.

Задание

1. Проанализируйте текущую реализацию MVP и определите точки интеграции аутентификации/авторизации: защищаемые endpoint'ы, пользовательские сценарии входа/выхода, операции, требующие ролевой проверки.

2. Спроектируйте целевую схему авторизации на основе двух токенов:

   2.1. access token (короткое время жизни) для доступа к API;
   2.2. refresh token (длительное время жизни) для обновления access token;
   2.3. политика истечения, обновления и отзыва токенов (logout, компрометация, повторный вход);

3. Реализуйте серверную часть в **FastAPI**:

   3.1. endpoints аутентификации (минимально: вход, обновление токена, выход, получение текущего пользователя);
   3.2. проверку access token на защищенных маршрутах через зависимости;
   3.3. валидацию и ротацию refresh token (при обновлении сессии);
   3.4. хранение и отзыв refresh token (или их идентификаторов) с учетом требований безопасности;
   3.5. проверку ролей/прав на уровне бизнес-операций в связке с моделью доступа из предыдущей лабораторной работы.

4. Примените архитектурные и проектные паттерны при реализации backend-логики. Минимально использовать:

    4.1. разделение на слои (API → service → repository);
    4.2. dependency injection для подстановки сервисов и проверок;

5. Реализуйте клиентскую часть в **React + TypeScript**:

    5.1. форму входа и завершения сессии;
    5.2. централизованное хранение состояния аутентификации;
    5.3. автоматическое обновление access token при его истечении (через interceptor/middleware);
    5.4. защиту маршрутов и UI-элементов по состоянию аутентификации и ролям;
    5.5. корректное поведение при невалидной/истекшей сессии (очистка состояния, перенаправление на вход).

6. Проведите проверку корректности работы механизма:

    6.1. вход с корректными и некорректными учетными данными;
    6.2. доступ к защищенным ресурсам с валидным и невалидным access token;
    6.3. обновление access token через refresh token;
    6.4. отзыв сессии при выходе и блокирование дальнейшего обновления;
    6.5. проверка ограничений ролей и запрет операций вне полномочий.

7. Убедитесь, что после внедрения механизма аутентификации и авторизации:

    7.1. существующая функциональность MVP сохранена;
    7.2. сессии обрабатываются предсказуемо и безопасно;
    7.3. клиентская и серверная части используют единые правила доступа.

Лабораторная работа №3 - Разработка пользовательских интерфейсов с фильтрацией и управление пользовательскими данными с интеграцией объектного хранилища.

Цели лабораторной работы

1. Освоить проектирование интерфейсов фильтрации, поиска, сортировки и пагинации для существующего MVP.
2. Реализовать согласованное управление пользовательскими данными на клиенте и сервере.
3. Интегрировать объектное хранилище для загрузки, хранения и выдачи пользовательских файлов.
4. Обеспечить корректность, безопасность и устойчивость обработки данных и файловых операций.

Задание

1. На основе ранее реализованного MVP определите сущности и сценарии, требующие:
   1.1. фильтрации и поиска;
   1.2. сортировки и постраничного вывода;
   1.3. привязки пользовательских файлов к данным системы.
2. Спроектируйте интерфейс фильтрации в **React + TypeScript**:
   2.1. реализуйте форму фильтров (минимум 3 параметра);
   2.2. добавьте поиск по ключевым полям;
   2.3. реализуйте сортировку и пагинацию;
   2.4. обеспечьте сохранение состояния фильтров при навигации (query params/router state).
3. Реализуйте серверную поддержку фильтрации в **FastAPI**:
   3.1. добавьте query-параметры для фильтрации, сортировки и пагинации;
   3.2. обеспечьте валидацию входных параметров;
4. Реализуйте управление пользовательскими данными:
   4.1. создание, редактирование, просмотр и удаление записей;
   4.2. валидацию полей на клиенте и сервере;
   4.3. обработку конфликтов и ошибок (некорректные данные, отсутствие прав, несуществующие записи);
   4.4. согласование бизнес-правил с механизмом авторизации и ролей из предыдущих лабораторных работ.
5. Интегрируйте объектное хранилище (S3-совместимое или облачное):
   5.1. загрузка файлов, привязка к сущностям приложения;
   5.2. получение файлов по защищенному механизму (например, pre-signed URL);

5.3. удаление файлов и очистка связанных метаданных;
5.4. ограничение допустимых типов и размеров файлов.

6. Реализуйте клиентскую часть работы с файлами:
   6.1. интерфейс загрузки (выбор, отправка, индикация статуса);
   6.2. отображение списка прикрепленных файлов;
   6.3. скачивание/просмотр файла;
   6.4. обработку ошибок загрузки и ограничений доступа.

7. Проведите проверку функциональности:
   7.1. корректность фильтрации, поиска, сортировки и пагинации;
   7.2. корректность CRUD-операций с пользовательскими данными;
   7.3. корректность загрузки, получения и удаления файлов в объектном хранилище;
   7.4. соблюдение ограничений доступа к данным и файлам;
   7.5. сохранение работоспособности существующей функциональности MVP после доработки.

Лабораторная работа №4 - SEO-оптимизация веб-приложения и интеграция сторонних API.

Цели лабораторной работы

1. Освоить практику SEO-доработки существующего MVP без изменения его бизнес-логики.
2. Реализовать техническую и контентную SEO-оптимизацию на клиентской и серверной частях.
3. Интегрировать сторонний API в архитектуру приложения с учетом надежности и безопасности.
4. Обеспечить корректную работу индексации, внешних интеграций и текущего функционала MVP.

Задание

1. На основе ранее реализованного MVP определите страницы и разделы, требующие SEO-оптимизации:

   1.1. публичные страницы, подлежащие индексации;
   1.2. служебные/закрытые страницы, которые должны исключаться из индексации;
   1.3. приоритетные страницы для поисковой выдачи.

2. Реализуйте базовую SEO-оптимизацию в вашем frontend приложении:

   2.1. корректную семантическую разметку (title, h1–h3, alt для изображений, структурированные блоки контента);
   2.2. человеко-понятные URL и единый подход к маршрутам;
   2.3. динамические мета-теги (title, description) для ключевых страниц;
   2.4. canonical URL для исключения дублей контента;
   2.5. Open Graph / social meta для предпросмотра ссылок.

3. Реализуйте техническую SEO-поддержку на стороне **FastAPI** и инфраструктуры:

   3.1. sitemap.xml для индексируемых маршрутов;
   3.2. robots.txt с правилами обхода;
   3.3. корректные HTTP-статусы для несуществующих и недоступных страниц (404, 403, 410 при необходимости);

3.4. базовую разметку структурированных данных (JSON-LD) минимум для одного типа страниц.

4. Выполните оптимизацию производительности страниц, влияющую на SEO:

   4.1. lazy loading для медиа и тяжелых компонентов;
   4.2. минимизацию избыточных клиентских запросов;
   4.3. оптимизацию размера передаваемых данных и статических ресурсов;
   4.4. проверку стабильности отображения контента при загрузке.

5. Интегрируйте один сторонний API в существующий MVP:

   5.1. определите сценарий использования внешних данных;
   5.2. реализуйте серверный слой интеграции (service/adapter) в FastAPI;
   5.3. настройте работу с ключами доступа через переменные окружения;
   5.4. добавьте обработку ошибок: таймауты, повторные попытки, ограничение частоты запросов;
   5.5. нормализуйте внешний ответ в формат, используемый приложением.

6. Реализуйте клиентскую часть для работы с внешними данными:
   6.1. отображение данных в UI;
   6.2. состояния загрузки, пустого результата и ошибки;
   6.3. корректное поведение при недоступности внешнего API (graceful degradation).

7. Проведите проверку результата:

   7.1. корректность генерации и доступности sitemap.xml и robots.txt;
   7.2. корректность мета-тегов и canonical на целевых страницах;
   7.3. корректность обработки отказов стороннего API;
   7.4. сохранение работоспособности существующего функционала MVP после доработки.

Лабораторная работа №5 - Комплексное тестирование клиентской и серверной частей веб-приложения.

Цели лабораторной работы

1. Освоить подход к комплексному тестированию существующего MVP с учетом доработок предыдущих лабораторных работ.
2. Реализовать модульное, интеграционное и сквозное тестирование клиентской и серверной частей.
3. Проверить корректность бизнес-логики, авторизации, работы с данными и внешними интеграциями.
4. Обеспечить воспроизводимость и автоматизацию проверки качества приложения.

Задание

1. На основе текущего состояния MVP сформируйте тестовую модель приложения:

   1.1. выделите критические пользовательские сценарии;
   1.2. определите ключевые бизнес-правила и ограничения;
   1.3. зафиксируйте области повышенного риска (аутентификация, роли/права, работа с файлами, интеграции API).

2. Реализуйте тестирование backend в **FastAPI**:

   2.1. модульные тесты сервисного слоя;
   2.2. интеграционные тесты endpoint’ов;
   2.3. проверку кодов ответа и структуры данных;
   2.4. проверку граничных случаев и ошибок валидации;

3. Реализуйте тестирование frontend в **React + TypeScript**:

   3.1. модульные тесты компонентов и утилит;
   3.2. тесты пользовательских сценариев (формы, фильтры, навигация, состояния загрузки/ошибок);
   3.3. проверку ролевого поведения интерфейса и защиты маршрутов;
   3.4. проверку корректной обработки серверных ошибок и истечения сессии.

4. Реализуйте сквозные (E2E) проверки основных бизнес-сценариев:

    4.1. вход/выход и восстановление сессии;
    4.2. выполнение операций CRUD по ролям;
    4.3. сценарии фильтрации/сортировки/пагинации;
    4.4. сценарии загрузки и получения файлов из объектного хранилища;
    4.5. сценарии работы со сторонним API при штатной работе и отказах.

5. Настройте тестовую инфраструктуру:

    5.1. отдельное тестовое окружение и конфигурации;
    5.2. изолированную тестовую БД/фикстуры;
    5.3. мокирование внешних зависимостей (сторонние API, объектное хранилище);
    5.4. очистку состояния между тестами.

6. Введите метрики качества тестирования:

    6.1. минимально контролируемое покрытие по backend и frontend;
    6.2. разделение быстрых и длительных тестов (unit/integration/e2e);
    6.3. единые правила именования и структуры тестов.

7. Проведите итоговую проверку:

    7.1. подтверждение корректной работы ключевых сценариев;
    7.2. подтверждение соблюдения прав доступа и ограничений безопасности;

Лабораторная работа №6 - Контейнеризация и автоматизация развертывания веб-приложения.

Цели лабораторной работы

1. Освоить контейнеризацию существующего решения для клиентской и серверной частей приложения.
2. Реализовать воспроизводимое окружение запуска и единый процесс сборки.
3. Настроить автоматизацию развертывания с проверками качества и контролем версий.
4. Обеспечить устойчивость, наблюдаемость и безопасную конфигурацию приложения в среде выполнения.

Задание

1. На основе текущего проекта подготовьте архитектуру контейнеризации:

    1.1. выделите сервисы приложения (frontend, backend, БД, reverse proxy, вспомогательные сервисы);
    1.2. определите сетевую схему взаимодействия контейнеров;

2. Реализуйте контейнеризацию компонентов:

    2.1. Подготовьте необходимые файлы для развертывания вашего приложения при помощи Angie/Nginx/Caddy ;
    2.2. создайте Dockerfile для **FastAPI**;
    2.3. настройте .dockerignore.

3. Настройте оркестрацию сервисов через docker-compose:

    3.1. запуск frontend, backend, БД и необходимых зависимостей одной командой;
    3.2. проброс портов, сети, тома и переменные окружения;
    3.3. healthcheck для ключевых сервисов;
    3.4. порядок запуска с учетом готовности зависимостей.

4. Обеспечьте безопасную и управляемую конфигурацию:

    4.1. вынесите конфигурацию в переменные окружения;
    4.2. исключите секреты из репозитория;

5. Автоматизируйте процесс развертывания (CI/CD):

    5.1. сборка и проверка образов при изменениях в репозитории;
    5.2. запуск обязательных тестов и линтеров перед публикацией;
    5.3. автоматическое развертывание целевого окружения после успешных проверок;

6. Проведите проверку итоговой конфигурации:

    6.1. воспроизводимость локального запуска и развертывания;
    6.2. корректность работы всех сервисов после контейнеризации;
    6.3. сохранение функциональности MVP и ранее реализованных доработок;
    6.4. устойчивость к типовым сбоям (падение сервиса, ошибка внешней зависимости, неуспешная миграция).